

LDD3, Ch1

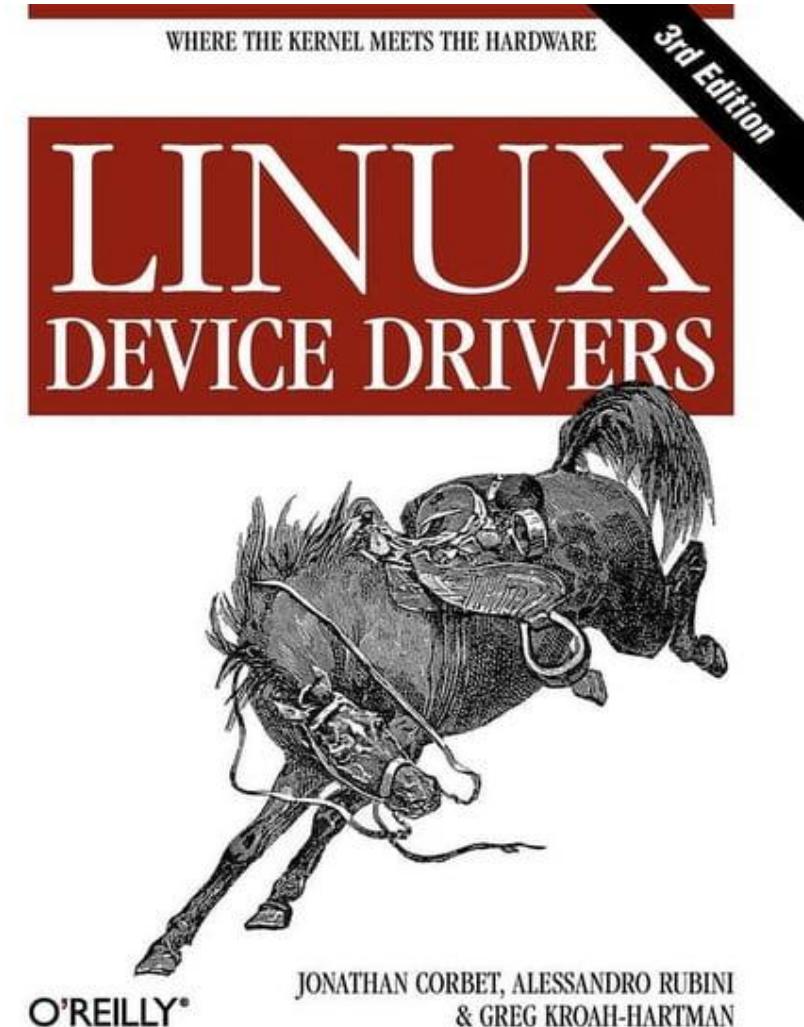
ECEN 427

BYU Electrical & Computer
Engineering

IRA A. FULTON COLLEGE OF ENGINEERING

Linux Device Drivers

- Lots of great content
- Available for free at <https://lwn.net/Kernel/LDD3/>
- Published Feb 2005
 - Booked uses 2.6 kernel
 - Linux currently at 7.2
 - Our PYNQ image uses Linux 6.6
- Keep in mind, many of the specific code examples (syntax) are out-of-date



Why do we care about writing device drivers?

“There are a number of reasons to be interested in the writing of Linux device drivers. The rate at which new hardware becomes available (and obsolete!) alone guarantees that driver writers will be busy for the foreseeable future...”

- The number of new hardware devices continues to rapidly grow...
 - Over 1.42 trillion semiconductor units shipped in 2024 (175 chips per person on earth, every year), up 10% from year before
 - 21 billion connected IoT devices in 2025, growing 14% a year, with roughly 39 billion forecast for 2030
 - Cheap FPGAs and custom SoCs mean "new hardware" now includes devices nobody else has ever built
 - 50+ companies are designing AI accelerator chips right now: Google TPU v7 Ironwood, Amazon Trainium 3, Microsoft Maia 200, Meta MTIA 300 (with MTIA 400 in test), and OpenAI's chip with Broadcom, Cerebras, Groq, SambaNova, Etched, MatX, Tenstorrent, Lightmatter, Neurophos



Policy & Mechanism

Mechanism vs Policy

As a programmer, you are able to make your own choices about your driver, and choose an acceptable **trade-off between the programming time required and the flexibility of the result.**

Though it may appear strange to say that a driver is “flexible”, we like this word because it emphasizes that the role of a device driver is **providing mechanism, not policy.**

The distinction between mechanism and policy is one of the best ideas behind the Unix design. Most programming problems can indeed be split into two parts:

- “what capabilities are to be provided” (the ***mechanism***), and
- “how those capabilities can be used” (the ***policy***).

If the two issues are addressed by different parts of the program, or even by different programs altogether, the software package is much easier to develop and to adapt to particular needs.

(Pages 2-3)

Mechanism vs Policy

Mechanism :

- What capabilities will be provided
- What can be done with the hardware

Policy

- How the capabilities will be used
- How to use the hardware in applications



Example: Disk Driver

Mechanism

Policy

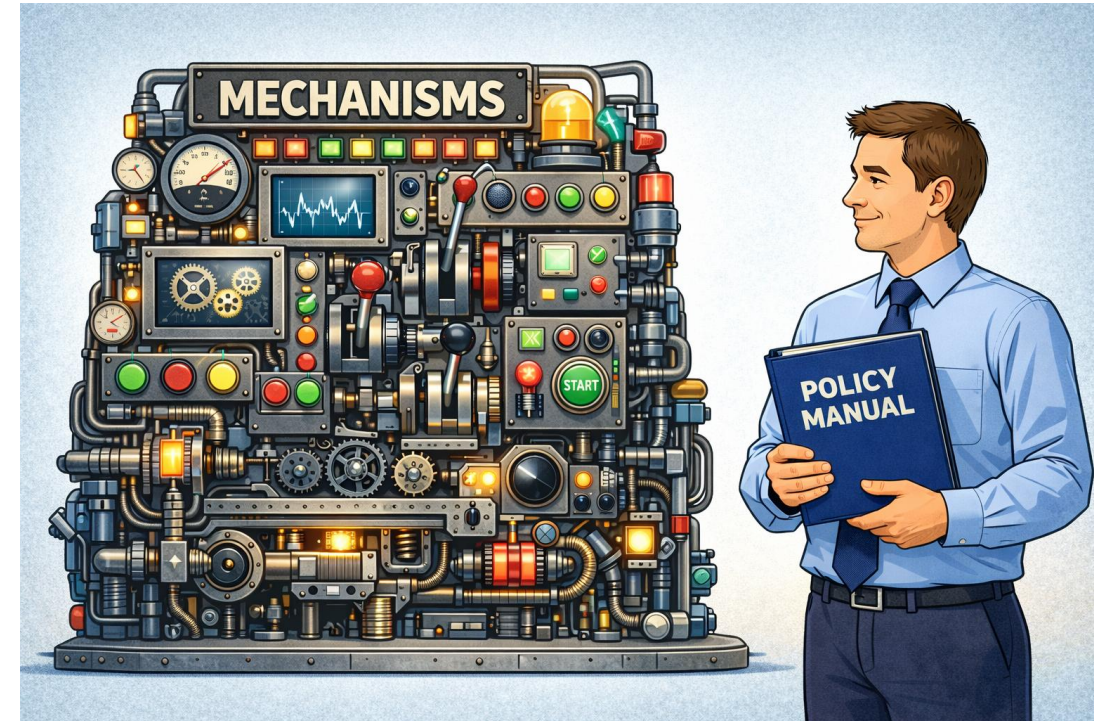
1. Read & write data
2. Error Handling
3. Bad Sector Handling
4. Data Caching
5. Power Management

Drivers Provide Mechanism, not Policy

When writing drivers, a programmer should pay particular attention to this fundamental concept: write kernel code to access the hardware, but don't force particular policies on the user, since different users have different needs.

The driver should deal with making the hardware available, leaving all the issues about how to use the hardware to the applications. A driver, then, is flexible if it offers access to the hardware capabilities without adding constraints.

(Page 3)



Policy & Mechanism in Lab 2 Drivers

Did we introduce any policy in our Lab 2 drivers?

Did we provide mechanism for all hardware features?

Policy-free drivers have a number of typical characteristics. These include support for both synchronous and asynchronous operation, the ability to be opened multiple times, the ability to exploit the full capabilities of the hardware, and the lack of software layers to “simplify things” or provide policy-related operations. Drivers of this sort not only work better for their end users, but also turn out to be easier to write and maintain as well.

...The actual driver design should be a balance between many different considerations. For instance, a single device may be used concurrently by different programs, and the driver programmer has complete freedom to determine how to handle concurrency...One major consideration is the trade-off between the desire to present the user with as many options as possible and the time you have to write the driver, as well as the need to keep things simple so that errors don't creep in.

(Page 3)

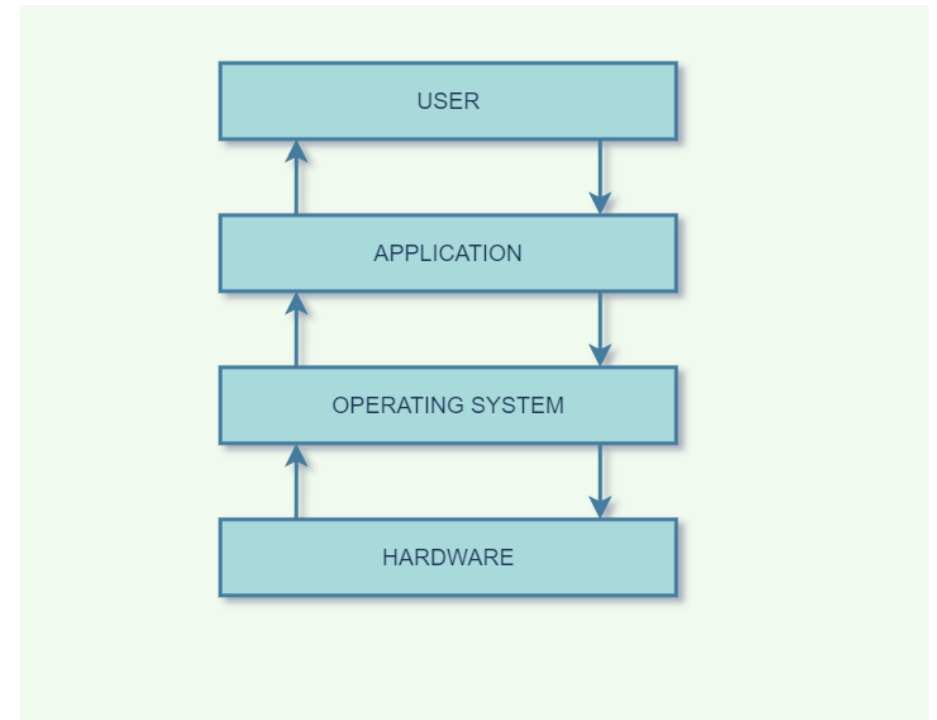
- What is synchronous vs asynchronous operation?
- What is concurrency? What sort of issues might arise when trying to make a concurrent driver?

- PYNQ UART example (opening /dev/ttyUSB1 multiple times)
 - Typically not permitted (screen opened in exclusive mode)
 - sudo bypasses the exclusive mode

The Linux Kernel

Recall: Kernel Responsibilities

1. Process management
2. Memory management
3. File system management
4. Device management
5. Network management



<https://www.mygreatlearning.com/blog/what-is-operating-system/>

Process management

The kernel is in charge of creating and destroying processes and handling their connection to the outside world (input and output). Communication among different processes (through signals, pipes, or interprocess communication primitives) is basic to the overall system functionality and is also handled by the kernel. In addition, the scheduler, which controls how processes share the CPU, is part of process management. More generally, the kernel's process management activity implements the abstraction of several processes on top of a single CPU or a few of them.

Memory management

The computer's memory is a major resource, and the policy used to deal with it is a critical one for system performance. The kernel builds up a virtual addressing space for any and all processes on top of the limited available resources. The different parts of the kernel interact with the memory-management subsystem through a set of function calls, ranging from the simple malloc/free pair to much more complex functionalities.

Filesystems

Unix is heavily based on the filesystem concept; almost everything in Unix can be treated as a file. The kernel builds a structured filesystem on top of unstructured hardware, and the resulting file abstraction is heavily used throughout the whole system. In addition, Linux supports multiple filesystem types, that is, different ways of organizing data on the physical medium. For example, disks may be formatted with the Linux-standard ext3 filesystem, the commonly used FAT filesystem or several others.

Device control

Almost every system operation eventually maps to a physical device. With the exception of the processor, memory, and a very few other entities, any and all device control operations are performed by code that is specific to the device being addressed. That code is called a device driver. The kernel must have embedded in it a device driver for every peripheral present on a system, from the hard drive to the keyboard and the tape drive. This aspect of the kernel's functions is our primary interest in this book.

Networking

Networking must be managed by the operating system, because most network operations are not specific to a process: incoming packets are asynchronous events. The packets must be collected, identified, and dispatched before a process takes care of them. The system is in charge of delivering data packets across program and network interfaces, and it must control the execution of programs according to their network activity. Additionally, all the routing and address resolution issues are implemented within the kernel.

Loadable Modules

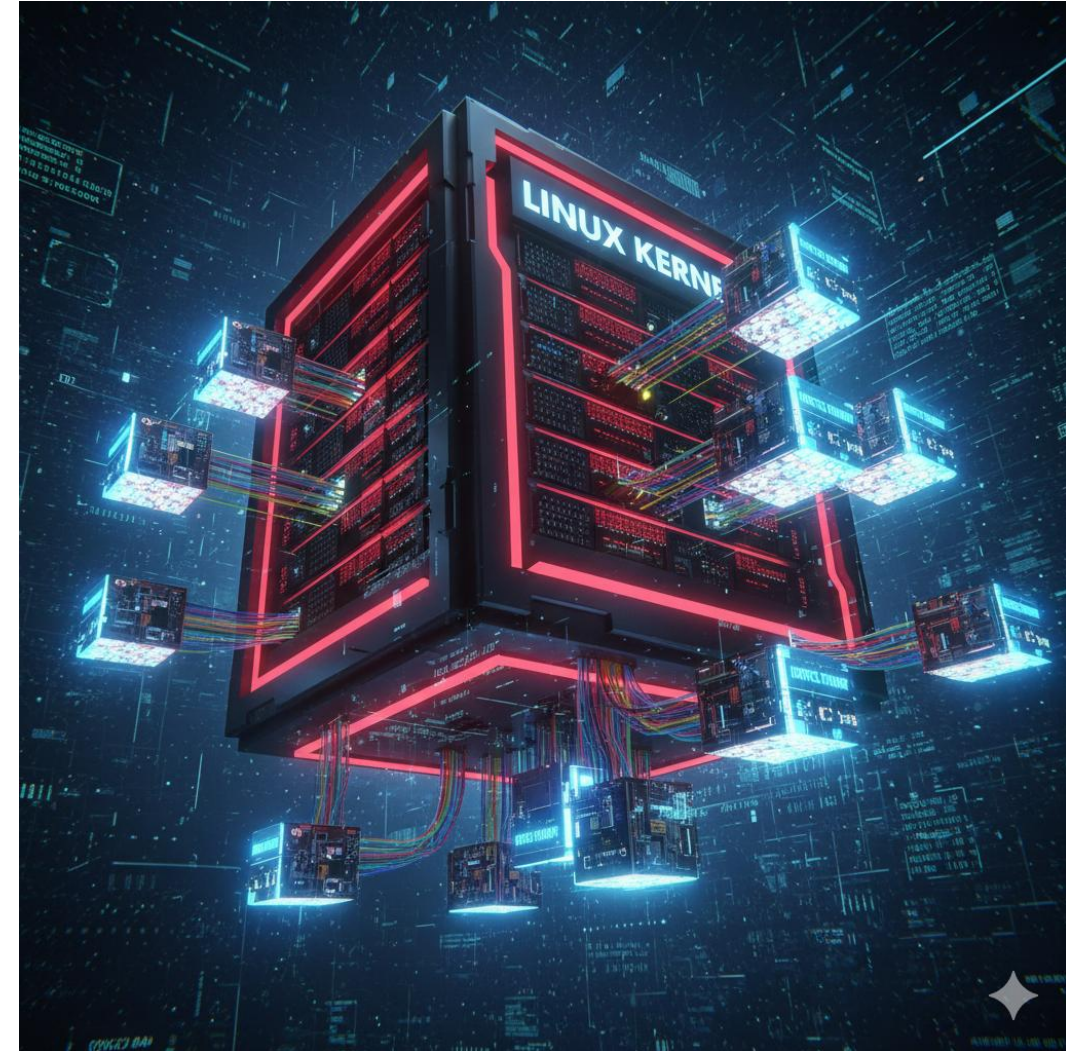
One of the good features of Linux is the ability to extend at runtime the set of features offered by the kernel.

Each piece of code that can be added to the kernel at runtime is called a **loadable module**.

The Linux kernel offers support for quite a few different types (or classes) of modules, **including, but not limited to, device drivers**.

Each module is made up of object code (not linked into a complete executable) that can be dynamically linked to the running kernel by the **insmod** program and can be unlinked by the **rmmod** program.

(Page 5)



What is the GPL?

- The GNU General Public License, written by Richard Stallman for the GNU project (v1 1989, v2 1991, v3 2007)
 - Linux is GPLv2
- A “copyleft” license. (*pun on "copyright" coined by Richard Stallman*)
 - You may use, copy, modify and redistribute the code freely, on one condition: if you distribute it, or a work derived from it, the **recipient gets the source under the same license**.
 - Contrast with permissive licenses (MIT, BSD, Apache): take the code, **close it**, ship it.
- You can change GPL code and keep it private forever. Only shipping it to someone else obligates you to give them the source.

Licensing: the kernel is GPL. Is your module?

- Linux is GPLv2. A module links directly against kernel internals, so most kernel developers consider it a derivative work.
- You must declare a GPL license (`MODULE_LICENSE("GPL")`) in your kernel module
 - Anything else taints the kernel, and the community will not look at your bug reports.
 - Non-GPL modules cannot call certain functions that are GPL-only
 - (without adopting a GPL license, we could not do Lab 5 as designed)
- The NVIDIA saga
 - 20+ years of a closed binary blob behind a thin open-source shim.
 - Linus, 2012: "the single worst company we've ever dealt with."
 - Linux 5.9 (2020) closed a GPL-shim loophole, it was a cat-and-mouse game for the next two years
 - 2022: NVIDIA open-sourced its kernel module, default since 2024.
 - Most of the logic moved into closed GPU firmware on a RISC-V CPU on the GPU device
 - The CUDA/OpenGL user space is still proprietary.
- User space drivers (Lab 2) sidestep all of this. The GPL stops at the system-call boundary.



Types of Device Drivers

#1 Character devices

A character (char) device is one that can be accessed as a stream of bytes (like a file); a char driver is in charge of implementing this behavior. Such a driver usually implements at least the open, close, read, and write system calls.

The text console (/dev/console) and the serial ports (/dev/ttyS0 and friends) are examples of char devices, as they are well represented by the stream abstraction.

Char devices are accessed by means of filesystem nodes, such as /dev/tty1 and /dev/lp0.

(Page 6)

#2 Block Devices

Like char devices, block devices are accessed by filesystem nodes in the /dev directory.

A block device is a **device (e.g., a disk) that can host a filesystem.**

In most Unix systems, a block device can only handle I/O operations that transfer one or more whole blocks, which are usually 512 bytes (or a larger power of two) bytes in length. Linux, instead, allows the application to read and write a block device like a char device—it permits the transfer of any number of bytes at a time.

As a result, block and char devices **differ only in the way data is managed internally by the kernel**, and thus in the kernel/driver software interface. Like a char device, each block device is accessed through a filesystem node, and the difference between them is transparent to the user. Block drivers have a completely different interface to the kernel than char drivers.

(Page 7)

#3 Network interfaces

Any network transaction is made through an interface, that is, a device that is able to exchange data with other hosts. Usually, an interface is a hardware device, but it might also be a pure software device, like the loopback interface. A network interface is in charge of sending and receiving data packets, driven by the network subsystem of the kernel, without knowing how individual transactions map to the actual packets being transmitted. Many network connections (especially those using TCP) are stream-oriented, but network devices are, usually, designed around the transmission and receipt of packets. A network driver knows nothing about individual connections; it only handles packets.

Not being a stream-oriented device, a network interface isn't easily mapped to a node in the filesystem, as `/dev/tty1` is. The Unix way to provide access to interfaces is still by assigning a unique name to them (such as `eth0`), but that name doesn't have a corresponding entry in the filesystem. Communication between the kernel and a network device driver is completely different from that used with char and block drivers. Instead of read and write, the kernel calls functions related to packet transmission.